# 🧪 ЛАБОРАТОРНАЯ РАБОТА №3

## Тема: *Компьютерная арифметика*

**Семестр:** 1
**Раздел:** Представление данных и арифметические операции

## 🎯 Цель работы:

Научиться:

- описывать простые арифметические устройства с использованием логических конструкций;
- реализовывать сумматоры, компараторы, схемы сравнения и переключения на логическом уровне;
- анализировать представление чисел в двоичном коде (прямом, обратном, дополнительном);
- работать со сдвиговыми операциями и уметь представлять числа в знаковом и беззнаковом формате.

## 📚 Теоретическая база:

- **Типы представлений чисел:** прямой, обратный, дополнительный коды.
- **Сумматоры:** полусумматор и полный сумматор.
- **Логические операции:** AND, OR, NOT, XOR.
- **Арифметические логические устройства (АЛУ):** схемы, реализующие арифметические и логические операции.
- **Понятие переполнения, переноса, знака в двоичной арифметике.**

В этой лабораторной работе мы переходим к представлению чисел и выполнению арифметических операций над ними. Когда группа битов объединяется в пучок в Logisim, она представляет собой многобитное значение — по сути, целое число в двоичной записи. При этом нумерация битов начинается с младшего разряда (бит 0), а сам вывод отображается в порядке от старшего к младшему биту.

Для представления только положительных целых чисел достаточно обычной двоичной записи. Но чтобы кодировать также и отрицательные значения, применяются различные системы представления: прямой код, обратный код, дополнительный код. Наиболее широко используется именно дополнительный код (two’s complement), поскольку он упрощает выполнение арифметических операций.

В восьмибитной системе дополнительного кода можно закодировать значения от -128 до 127. Положительные числа (и ноль) имеют в старшем бите “0”, отрицательные — “1”. Чтобы получить число, противоположное по знаку, применяется следующий алгоритм в начале инвертируем число, потом к получившемуся результату прибавляем единицу.

| Двоичное число | Десятичное число |
|---|---|
| 1000 0000 | -128 |
| 1000 0001 | -127 |
| 1000 0010 | -126 |
| 1000 0011 | -125 |
| … | … |
| 1111 1110 | -2 |
| 1111 1111 | -1 |
| 0000 0000 | 0 |
| 0000 0001 | 1 |
| 0000 0010 | 2 |
| … | … |
| 0111 1100 | 124 |
| 0111 1101 | 125 |
| 0111 1110 | 126 |
| 0111 1111 | 127 |

Преимущество дополнительного кода в том, что арифметические операции (сложение, вычитание) выполняются одинаково для знаковых и беззнаковых чисел. Например, сложение 00000001 и 10000001 даёт 10000010. Интерпретируя операнды и результат как беззнаковые числа, получаем 1 + 129 = 130. Интерпретируя как знаковые (в доп. коде): 1 + (-127) = -126. Результат тот же — корректный.

Если требуется изменить разрядность числа (например, из 8 бит в 16), нужно корректно расширить значение. Для этого применяется компонент "Расширитель битов" с режимом "Знак". Это особенно важно при переходе между компонентами с разной разрядностью.

Сложение выполняется с учётом бита переноса (carry bit). Полный одноразрядный сумматор (1-bit full adder) принимает три входа (два операнда и бит переноса), и выдаёт сумму и новый бит переноса. Объединяя такие сумматоры последовательно, можно реализовать многоразрядный сумматор с последовательным переносом. Его главный недостаток — большая задержка.

Для ускорения сложения применяется сумматор с параллельным переносом (carry look-ahead adder), использующий сигналы G (generate) и P (propagate). Эти сигналы позволяют рассчитать все биты переноса одновременно, что значительно снижает задержку.

Вычитание можно реализовать двумя способами: через одноразрядные вычитатели с битами займа (borrow), либо через сложение с отрицательным значением (X - Y = X + (-Y)). Второй способ проще в реализации при наличии компонента "Отрицатель".

Умножение выполняется по классическому алгоритму "столбиком": частичные произведения сдвигаются и складываются. В Logisim можно реализовать умножитель, который работает только с беззнаковыми числами. Разрядность результата в два раза больше исходной.

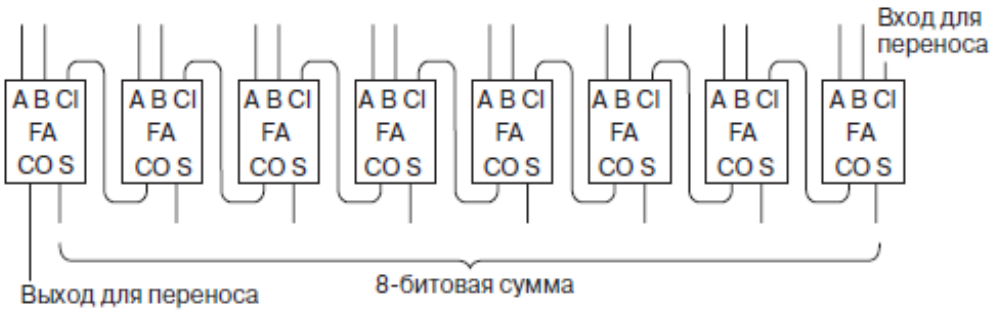
Деление в этой лабораторной не рассматривается, так как требует сложной реализации. Однако важно помнить, что результат деления — это два значения: частное и остаток, которые обычно выводятся на разные выходы.

Работа с числами с плавающей точкой в этой лабораторной не рассматривается, однако можно изучить эту тему дополнительно и попробовать реализовать простейшее устройство самостоятельно.

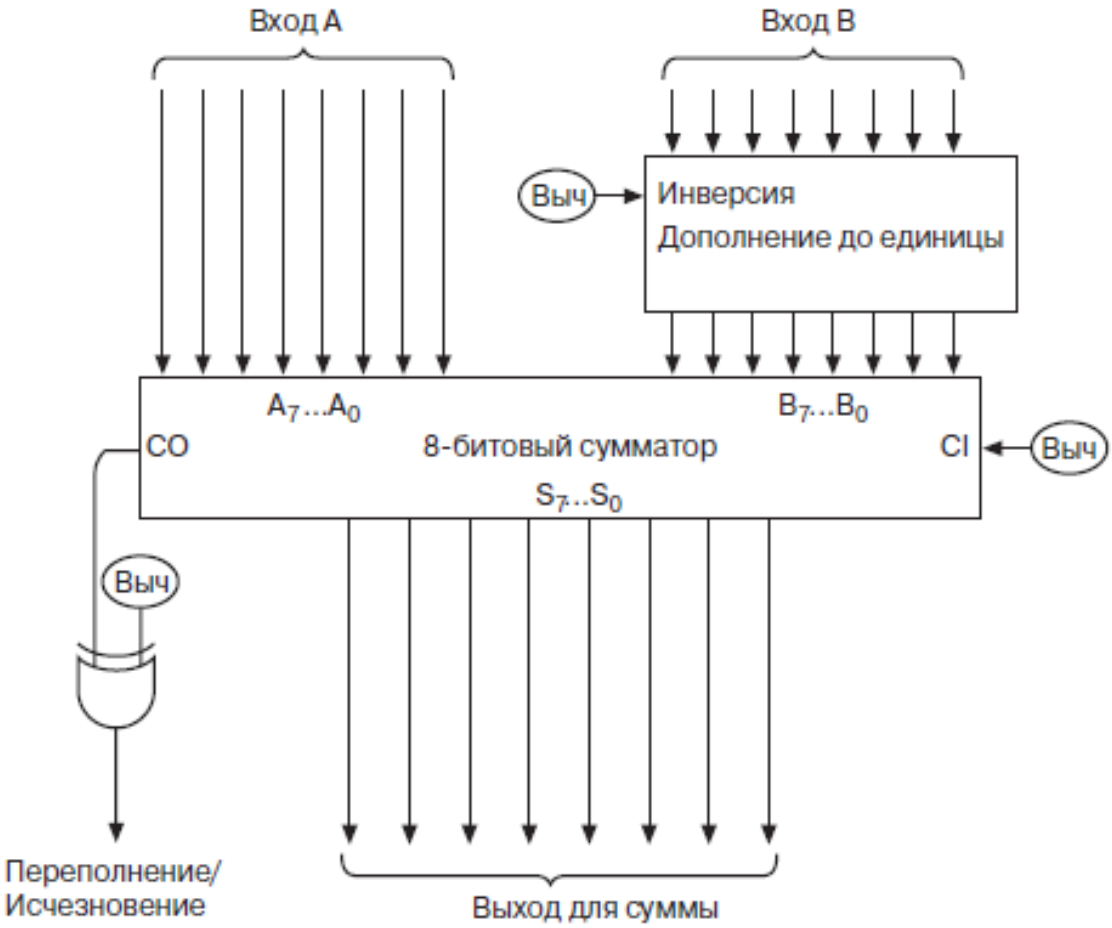
Все рассмотренные схемы являются комбинационными логическими устройствами и могут быть реализованы на логических элементах.

**Задание 1.** Спроектируйте полный одноразрядный сумматор в среде Logisim. Устройство должно принимать на вход два операнда (A и B), а также бит переноса (Cin), и выдавать сумму (S) и бит переноса в следующий разряд (Cout).

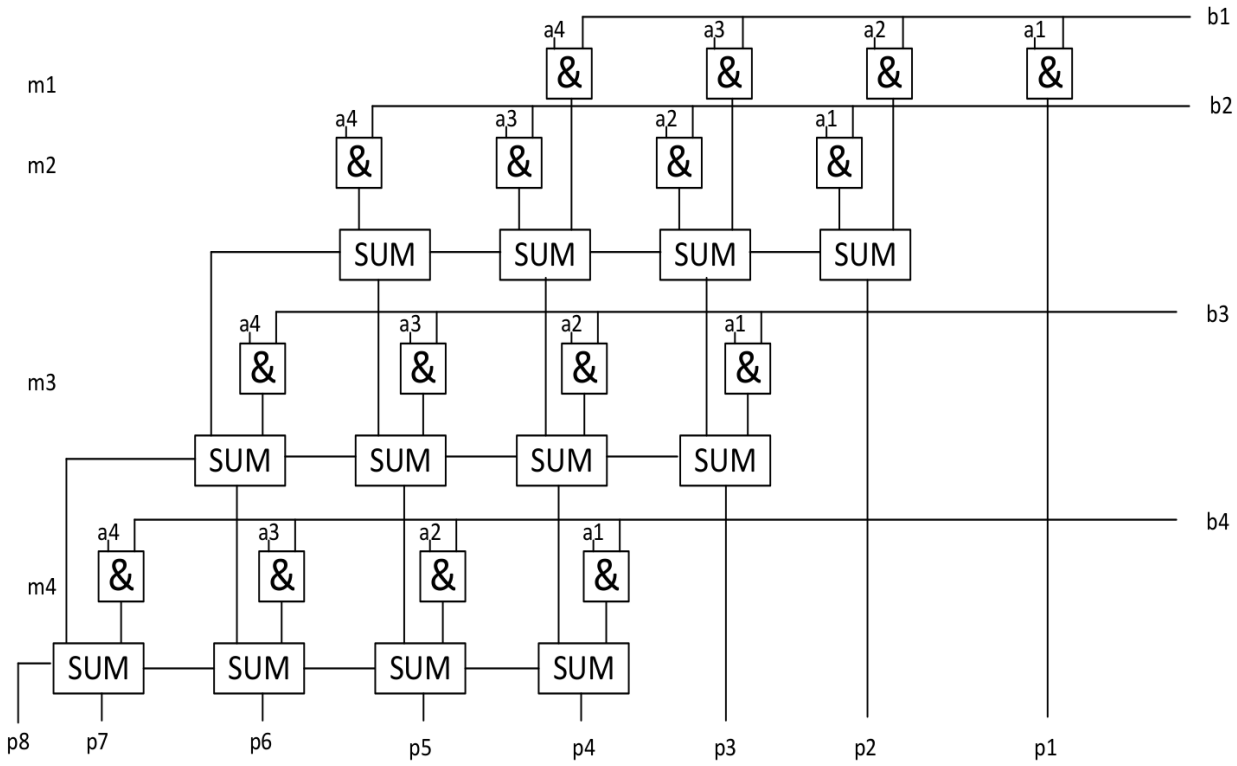
**Задание 2.** На основе схемы из задания 1 постройте 8-разрядный сумматор с последовательным переносом (ripple-carry adder). Все одноразрядные сумматоры должны быть соединены по цепочке через переносы.




**Задание 3.** Постройте 8-разрядный вычитатель, реализованный через сложение с отрицанием. Используйте сумматор из задания 4 и отрицатель из задания 8. На последнем выходе Cout реализуйте флаг указывающий на переполнение при сложении и на отрицательное значение при вычитании



**Задание 4.** Спроектируйте устройство умножения беззнаковых чисел. Входы — два 4-разрядных множителя, выходы —8-разрядная шина.




**Задание 5.** Провести исследования сдвигов чисел, а полученные результаты представить в виде знакового и без знакового десятичного числа. Выявить закономерности и ограничения к применимости сдвигов к проведению арифметических операций (делению и умножению).

| | | Логический сдвиг | | Арифметический сдвиг | | Циклический сдвиг | |
|---|---|---|---|---|---|---|---|
| | | Влево | Вправо | Влево | Вправо | Влево | Вправо |
| Положительное число | Четное | знаковый /беззнаковый | знаковый /беззнаковый | знаковый /беззнаковый | знаковый /беззнаковый | знаковый /беззнаковый | знаковый /беззнаковый |
| | Нечетное | знаковый /беззнаковый | знаковый /беззнаковый | знаковый /беззнаковый | знаковый /беззнаковый | знаковый /беззнаковый | знаковый /беззнаковый |
| Отрицательное число | Четное | знаковый /беззнаковый | знаковый /беззнаковый | знаковый /беззнаковый | знаковый /беззнаковый | знаковый /беззнаковый | знаковый /беззнаковый |
| | Нечетное | знаковый /беззнаковый | знаковый /беззнаковый | знаковый /беззнаковый | знаковый /беззнаковый | знаковый /беззнаковый | знаковый /беззнаковый |

### Общие требования к выполнению лабораторной работы №3

- **Единый проект.** Все задания лабораторной работы необходимо выполнить в одном `.circ`-файле проекта Logisim.
- **Структурированность.** Каждое логическое устройство должно быть реализовано как отдельная подсхема (circuit) с понятным и осмысленным названием.
- **Именование входов и выходов.** Все входные и выходные контакты в подсхемах должны быть подписаны, отражая их назначение (например: `A`, `B`, `Cin`, `Sum`, `Cout` и т.п.).
- **Тестовая схема.** Для проверки правильности работы всех разработанных устройств создаётся отдельная **тестовая схема**, содержащая тестовые примеры для всех заданий.
- **Демонстрация работы.** Для защиты лабораторной работы студент должен продемонстрировать корректность функционирования **каждого устройства** на **нескольких примерах входных данных**, отображаемых на тестовой схеме.

## Вопросы для защиты лабораторной работы №3

**Тема:** Компьютерная арифметика

1. Что такое прямой, обратный и дополнительный коды?
2. В чем отличие между прямым, обратным и дополнительным кодами?
3. Почему в современных компьютерах для представления отрицательных чисел используется дополнительный код?
4. Как определить знак числа в двоичном представлении?
5. В каком диапазоне представляются числа в 8-битной системе с дополнительным кодом?
6. Как получить отрицательное число из положительного в дополнительном коде?
7. Что происходит при сложении положительного и отрицательного чисел в дополнительном коде?
8. Что такое бит переноса (carry bit) и как он используется при сложении?
9. Что понимается под переполнением при арифметических операциях?
10. Как определить наличие переполнения при сложении двух знаковых чисел?

11. Чем отличается полусумматор от полного сумматора?
12. Какие логические операции используются для построения сумматора?
13. Какие входы и выходы имеет одноразрядный полный сумматор?
14. Как соединяются сумматоры при построении 8-разрядного сумматора с последовательным переносом?
15. Почему в 8-битном сумматоре может возникнуть ошибка при интерпретации результата как знакового числа?
16. Как можно реализовать вычитание через сложение с отрицанием?
17. Почему при сложении и вычитании в дополнительном коде можно использовать одну и ту же схему?
18. Можно ли реализовать вычитание, не используя блок отрицания? Как?
19. Почему важно правильно именовать входы и выходы при проектировании логических схем?
20. Как использование параллельного переноса ускоряет выполнение сложения?

21. Почему результат умножения двух 4-разрядных чисел имеет 8 бит?
22. Как можно реализовать умножение в Logisim, если нет готового блока?

23. Чем отличается умножение знаковых и беззнаковых чисел в двоичной арифметике?

24. Что представляет собой операция сдвига в двоичной арифметике?
25. В чем различие между логическим, арифметическим и циклическим сдвигами?
26. Как выполняется логический сдвиг влево и как изменяется значение числа при этом?
27. Что происходит при логическом сдвиге вправо и чем он опасен при работе со знаковыми числами?
28. Как выполняется арифметический сдвиг влево, и отличается ли он от логического?
29. Почему при арифметическом сдвиге вправо сохраняется старший (знаковый) бит?
30. Что произойдет, если применить арифметический сдвиг вправо к отрицательному числу в дополнительном коде?
31. Что происходит при циклическом сдвиге влево?
32. Какова разница между циклическим и арифметическим сдвигами?
33. Как выполняется циклический сдвиг вправо и что происходит с битом, «выпавшим» из младшего разряда?
34. Можно ли при циклическом сдвиге потерять информацию? Почему?
35. Чем логический сдвиг влево похож на умножение на 2, а сдвиг вправо — на деление на 2?
36. Почему сдвиги не всегда могут заменить операции умножения и деления?
37. Как определить, при каких условиях логический сдвиг влево приводит к переполнению?
38. Почему при работе со знаковыми числами арифметический сдвиг безопаснее логического?
39. В каком случае результат логического сдвига и арифметического сдвига влево совпадает?
40. Как изменится значение числа $10011010_2$ (в дополнительном коде) после арифметического сдвига вправо на 1 бит?
41. Как проверить в Logisim, что результат арифметического сдвига действительно сохраняет знак числа?
42. Как реализовать схему, выполняющую выбор типа сдвига (логический / арифметический / циклический)?
43. Почему важно различать сдвиги при работе с беззнаковыми и знаковыми числами?
44. Как сдвиги влияют на разрядность числа?
45. Что произойдет, если применить циклический сдвиг к двоичному числу, состоящему только из единиц?
46. Можно ли использовать циклический сдвиг для генерации псевдослучайных последовательностей?
47. Как можно определить, изменился ли знак числа после арифметического сдвига?
48. Как используются сдвиги для ускорения вычислений в процессорах?
49. Что произойдет, если выполнить логический сдвиг вправо на количество бит, равное разрядности числа?
50. Можно ли построить универсальный модуль «Сдвигатель» (Barrel Shifter)? Какие типы сдвигов он должен поддерживать?
51. Чем отличается реализация циклического сдвига влево и вправо с точки зрения схемотехники?
52. Как проверить корректность работы схемы сдвига в Logisim — какие тесты стоит использовать?

53. Объясните, почему операция вычитания через дополнительный код упрощает реализацию арифметических схем.

54. Почему для некоторых операций (например, деления) важно различать знаковое и беззнаковое представление чисел?
55. Как можно визуально продемонстрировать в Logisim эффект переполнения?
56. Как можно модернизировать 8-разрядный сумматор, чтобы он автоматически определял переполнение?
57. Реализуемо ли умножение со знаковыми числами в Logisim без встроенных компонентов?
58. Почему в некоторых случаях выгоднее использовать арифметические сдвиги вместо операций умножения или деления?
59. Можно ли объединить все реализованные устройства в единую тестовую схему и какие преимущества это даёт?
60. Как по тестовым результатам определить корректность работы всех реализованных схем?